

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab7-MPI 并行应用

姓 名 林宏宇

学 号 22320001

学 院 计算机学院

专 业 计算机科学与技术

2026 年 5 月 18 日

1 实验目的

- 验证并行 FFT 的加速效果与可扩展性
- 评估数据打包对通信性能的优化作用
- 分析并行规模对内存消耗的影响

2 实验内容

- **串行 FFT 分析与并行化改造**：
  - 阅读并理解提供的串行傅里叶变换代码 (fft_serial.cpp)。
  - 使用 MPI 对串行 FFT 代码进行并行化改造。
- **MPI 数据通信优化**：
  - 应用 MPI 数据打包技术（`MPI_Pack`/`MPI_Unpack`）来对通信数据进行重组，以优化消息传递效率。
- **程序性能与内存分析**：
  - 分析在不同并行规模及不同问题规模条件下，并行 FFT 程序的加速比和并行效率。
  - 分析数据打包技术对于并行程序整体性能的具体影响。
  - 使用 Valgrind Massif 工具采集并分析并行程序在不同配置下的内存消耗。

3 实验环境

- **CPU**：Intel Xeon Gold 6240R @ 2.40 GHz（最大睿频 4.00 GHz），Cascade Lake 微架构，支持 AVX-512 指令集
- **核心与线程**：2 路 NUMA 节点（NUMA node0: CPU 0–23, 48–71；NUMA node1: CPU 24–47, 72–95），每路 24 核 48 线程，共 48 物理核 / 96 逻辑线程（超线程开启），本次实验仅使用物理核范围内的进程数（$P \leq 16$），未触及超线程
- **Cache 层次**：L1d/L1i: 32 KB/核（共 1.5 MB）；L2: 1024 KB/核（共 48 MB）；L3: 35.75 MB/NUMA 节点（共 71.5 MB）

4 实验结果与分析

4.1 并行 FFT 的实现与正确性验证

算法设计：

本实验基于 Cooley-Tukey 按频率抽取（DIF）蝶形算法实现并行 FFT。串行核心函数为 `step`，该函数在每一阶段（参数 $mj = 1, 2, 4, \ldots, N/2$）执行蝶形运算：将输入数组分为前半部分 a（全局复位置 $0 \ldots N/2-1$）和后半部分 b（全局复位置 $N/2 \ldots N-1$），在每个蝶形组内计算 $a+b$（和值）与 $W \cdot (a-b)$（差值乘以旋转因子），分别输出至结果数组的上下两部分。

并行化策略：

采用一维块状数据划分。N 个复数等分为 P 份，进程 r 持有 N/P 个连续的复数（即 $2N/P$ 个双精度浮点数），覆盖全局复位置 $[r \cdot N/P,\ (r+1) \cdot N/P)$。对于所有 FFT 阶段，前半部分数据始终分布在进程 $0 \sim P/2-1$，后半部分数据始终分布在进程 $P/2 \sim P-1$。

每阶段通信与计算的三步流程如下：

1. **数据交换**：进程 r 与伙伴进程 $r \oplus (P/2)$ 通过 MPI_Sendrecv 交换其持有的全部 N/P 个复数。交换后双方均拥有对应 a 和 b 位置上的完整蝶形输入对。
2. **局部计算**：每个进程遍历其 a-位置集合（共 N/P 个，P=1 特殊处理为 $N/2$ 个），对每个位置计算蝶形运算，得到和值与差值两个输出值。
3. **输出重分布**：使用 MPI_Alltoallv 将计算得到的输出值按目标全局位置发送至正确进程，确保输出数组保持块状分布以供下一阶段使用。

该方案要求 P 为 2 的幂且 N/P 为整数。P=1 时无需进程间通信，直接使用串行逻辑。

正确性验证：

通过恒等式 $\mathrm{FFT}(\mathrm{FFT}(X)) = N \cdot X$ 检验。对所有测试配置（$N = 2^{16}, 2^{18}, 2^{20}$，$P = 1, 2, 4, 8, 16$），计算误差均在 10^{-16} 量级，与串行版本完全一致，验证了并行实现的正确性。

4.2 数据打包优化

实现方法：

在数据交换阶段应用 `MPI_Pack`/`MPI_Unpack` 进行通信优化。打包版本先将待发送的 $2N/P$ 个双精度数通过 `MPI_Pack` 序列化到连续字节缓冲区，再以 `MPI_PACKED` 数据类型进行 `MPI_Sendrecv` 传输，接收方通过 `MPI_Unpack` 反序列化。非打包版本直接以 `MPI_DOUBLE` 类型传输原始数组。两版本使用统一的 `use_pack` 标志切换，除数据交换步骤外其余逻辑完全一致。

需要指出，本实现中交换的数据本身已是连续内存块，因此 `MPI_Pack` 理论上并不必要（打包/解包引入了额外的内存拷贝开销）。实际观察到的性能差异反映了 MPI 实现层面对 `MPI_PACKED` 与 `MPI_DOUBLE` 数据类型的不同内部处理路径。

4.3 性能分析

4.3.1 不同问题规模 (N) 和并行规模 (P) 下的运行时间

表1展示了非打包版本在不同 N 和 P 组合下的原始运行时间。注意：不同 N 使用了不同的迭代次数（N=2^{16}: 100 次，N=2^{18}: 20 次，N=2^{20}: 5 次），因此绝对时间仅在同一 N 行内可比。加速比分析使用 P=1 的 MPI 版本作为基准（非纯串行版本）。

表 1: 不同问题规模 (N) 和并行规模 (P) 下的运行时间 (单位：秒)

问题规模 (N)	并行规模 (进程数 P)				
	P=1(基准)	P=2	P=4	P=8	P=16
$N = 2^{16}$	2.237	5.282	6.357	2.920	1.571
$N = 2^{18}$	2.023	6.144	6.259	2.801	1.386
$N = 2^{20}$	2.362	8.530	8.866	4.103	1.686

4.3.2 加速比与并行效率分析

表2展示了加速比 $S_p = T_1/T_p$ 和并行效率 $E_p = S_p/P$。

表 2: 不同配置下的加速比 (S_p) 与并行效率 (E_p)

N	指标	并行规模 (进程数 P)			
		P=2	P=4	P=8	P=16
2^{16}	S_p	0.42	0.35	0.77	1.42
	E_p	0.21	0.09	0.10	0.09
2^{18}	S_p	0.33	0.32	0.72	1.46
	E_p	0.16	0.08	0.09	0.09
2^{20}	S_p	0.28	0.27	0.58	1.40
	E_p	0.14	0.07	0.07	0.09

加速比分析：

- P=2 和 P=4 时加速比均小于 1（严重减速），总执行时间远超 P=1。小规模并行

下，进程间通信开销（MPI_Sendrecv 数据交换 + MPI_Alltoallv 输出重分布）远超计算并行化带来的收益。

- P=8 时加速比为 0.58–0.77，虽仍低于 1 但明显改善，说明通信开销占比开始下降。
- P=16 时加速比达到 1.40–1.46，首次超过基准，也是本次实验中唯一实现正向加速的配置。但考虑到投入了 16 倍的计算资源，1.4–1.5 倍的加速比远低于线性加速的理想预期。
- 加速比随 N 变化不大（P=16 时 1.40–1.46），因为本实现中每进程的计算量与通信量均随 N/P 等比增长，计算通信比近似恒定。

并行效率分析：

并行效率 $E_p = S_p/P$ 在所有配置下均处于 0.07–0.21 的低水平，说明投入的并行资源绝大部分被通信开销消耗。E_p 呈近似 $1/P$ 的衰减态势（P=2 时 0.14–0.21，P=16 时 0.09），表明当前实现的可扩展性存在结构性瓶颈。

性能局限性的深层分析（与理论预期的差距）：

本实现中通信复杂度为 $O((N/P)\cdot\log_2 N)$ 且全程需要 $2\log_2 N$ 次全局通信（以 $N = 2^{20}$ 为例，单次 FFT 需 40 次 MPI 集体操作）。相比经典高性能并行 FFT 实现（如转置法、六步 FFT 等）通常仅在少数阶段进行全局数据重分布，本实验实现由于在每个 FFT 阶段都执行全局通信，因此通信次数明显偏多，导致整体通信开销较高。造成这一差距的根本原因有两点：

1. **每阶段双通信模式**：本实现在每个 FFT 阶段都执行 MPI_Sendrecv（数据交换）和 MPI_Alltoallv（输出重分布），无法利用「前 $\log_2(N/P)$ 个阶段蝶形对完全位于进程内部」这一性质。这是因为串行代码采用按频率抽取（DIF）变体，所有蝶形对均跨越数组前半部分与后半部分的边界，而这两部分始终分布在不同的进程组中（进程 $0 \sim P/2-1$ 持有前半部分，$P/2 \sim P-1$ 持有后半部分）。因此即使是最早的阶段（$mj = 1$）也需要跨进程通信。
2. **通信-计算无法重叠**：当前 MPI_Sendrecv 和 MPI_Alltoallv 均为阻塞操作，通信期间进程闲置等待，无法利用非阻塞通信实现计算与数据传输的重叠。

优化路径分析（不改变报告既有数据，仅供讨论）：

若对算法进行以下结构性改进，理论上可将 P=16 时的加速比从 1.4 倍提升至接近线性加速：

- **采用转置法（Transpose FFT）**：将 N 点 FFT 视为二维 $\sqrt{N}\times\sqrt{N}$（或更一般地 $N/P \times P$）网格，先在各行做局部 FFT，通过一次 MPI_Alltoall 转置，再在各列做局部 FFT。全程仅需 1 次 Alltoall（前向 FFT），相比本实现的每阶段双通信（$2\log_2 N = 40$ 次），通信次数从 $O(\log N)$ 降至 $O(1)$。

- **引入按时间抽取（DIT）变体**：DIT 变体的蝶形配对距离为 mj（而非固定 $N/2$），前 $\log_2(N/P)$ 个阶段（$mj \leq N/P$）的所有蝶形对完全位于进程内部，无需任何通信。仅最后 $\log_2 P$ 个阶段需要跨进程通信（二进制交换法，每阶段 1 次 Sendrecv），总通信阶段数从 $\log_2 N$ 降至 $\log_2 P$（当 $P = 16$ 时从 20 降至 4）。
- **使用非阻塞通信**：以 MPI_Isend/MPI_Irecv 替代 MPI_Sendrecv，在等待数据传输完成的同时执行不依赖通信数据的局部计算，实现计算与通信的重叠。
- **消除重复计算**：当前实现中进程 r 与其伙伴 $r \oplus P/2$ 计算了完全相同的蝶形对（两者交换后拥有相同的数据），导致计算量翻倍。重构计算分配可使总计算量减半。

本实验的核心价值在于：通过真实数据定量地展示了通信设计对并行程序性能的决定性影响——即使算法在数值上正确，不当的通信模式也可能使并行化失去实际意义。这一反面案例恰好印证了并行计算课程的核心原则：**并行效率取决于通信最小化，而非仅并行化本身**。

4.3.3 数据打包对性能的影响

表3展示了所有并行规模下打包与非打包版本的完整性能对比。

表 3: 数据打包对运行时间的影响 (单位：秒)

N	P=2		P=4		P=8		P=16	
	非打包	打包	非打包	打包	非打包	打包	非打包	打包
2^{16}	5.282	5.609	6.357	6.456	2.920	2.272	1.571	1.553
2^{18}	6.144	6.526	6.259	6.713	2.801	2.285	1.386	1.384
2^{20}	8.530	8.927	8.866	8.873	4.103	4.213	1.686	1.683

从表3可以得出以下结论：

- **P=2,4（打包负面影响）**：在所有 N 下打包版本均比非打包版本慢。P=2 时打包带来约 5–6% 的性能下降，P=4 时下降 1–7%。这是因为在进程数较少时每条消息较大（$2N/P$ 个双精度数），数据本身已是连续内存块，`MPI_Pack/Unpack` 引入的额外内存拷贝开销是纯粹的负担，没有任何通信效率补偿。
- **P=8（打包条件性获益）**：在 N=2^{16} 和 2^{18} 下打包版本分别快 22% 和 18%，但在 N=2^{20} 时反而慢 2.7%。这一非单调行为说明 `MPI_PACKED` 与 `MPI_DOUBLE` 在 MPI 内部有不同的处理路径，在特定消息大小范围内（本实验中约 32–128 KB，对应 $N/P = 2^{13} \sim 2^{15}$ 个复数）打包路径可能有更好的内部缓冲策略。当消息继续增大（N=2^{20}, P=8 时约 256 KB），打包/解包的计算开销重新超过内部优化收益。

- **P=16（打包无明显影响）**：所有 N 下打包与非打包性能差异在 2% 以内，处于测量误差范围。此时每条消息很小（$N/P = 2^{12} \sim 2^{16}$ 个复数，即 8–128 KB），打包/解包的 CPU 开销与 MPI 内部处理差异的收益大致抵消。
- **总体结论**：对于本实验中的连续数据交换场景，MPI_Pack/Unpack 并非普适优化手段。其效果高度依赖于进程数（决定消息大小）和 MPI 实现细节，在部分配置下可能带来提升，也可能引入额外开销。对于非连续数据或需自定义数据布局的场景，打包技术可能有更大的实际价值。

4.4 内存消耗分析 (Valgrind Massif)

使用 Valgrind Massif 工具（带`--stacks=yes` 参数）对 N=4096 (2^{12}) 的并行 FFT 程序进行内存分析。

图 1: P=1 时内存消耗图 (ms_print 输出, N=4096)

```
    MB
3.610^   #
     |   #@:@@:::::::::::::::::@:::::@:::@:::::::::::@@::::@::
     |   #@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     |  @#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     | :@#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
     | :@#@:@@:::::::::::::::  ::@: :::@:::@:::::::::::@ ::::@::
   0 +-------------------------------------------------------->Mi
     0                                                   708.6
Number of snapshots: 59
Detailed snapshots: [2, 3 (peak), 5, 7, 8, 25, 30, 34, 47, 54]
```

选择较小的 N 是为了使 Valgrind（会使程序运行速度减慢 10–50 倍）能够在合理时间内完成采集。图1和图2分别展示了 P=1 和 P=4 时 `ms_print` 生成的内存消耗图。

图中 x 轴为程序运行时间（单位：百万指令 Mi），y 轴为内存消耗量（单位：MB），`#` 标记峰值快照，`@` 标记详细快照，`:` 标记普通快照。

图 2: P=4 时内存消耗图 (ms_print 输出, N=4096, 进程 47815)

```
    MB
3.532^                                                    :
     |    #:::::::::::::::::::::::::::::::::::::::::::::::@:::@:
     |    #: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |   @#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     |  :@#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     | ::@#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
     | ::@#: ::: :::::::::: :::::::::: :: : ::::::: :::@: :@:
   0 +-------------------------------------------------------->Mi
     0                                                    524.3
Number of snapshots: 53
Detailed snapshots: [3, 4 (peak), 46, 49]
```

表4总结了峰值快照的关键内存数据。

表 4: 峰值快照内存消耗数据 (N=4096, `--stacks=yes`)

配置	快照编号	时刻 (Mi)	总内存 (B)	有用堆 (B)	栈内存 (B)
P=1	3 (peak)	42.66	3,785,512	3,603,619	1,792
P=4 (进程 1)	4 (peak)	46.66	3,701,712	3,516,891	1,792

内存分配来源分析：

峰值时刻的 Massif 分配树显示：

- P=1：有用堆内存中 42.69%（约 1.54 MB）来自 MPI 内部通信缓冲区的内存池函数 `opal_free_list_grow_st`（属于 OpenMPI 的 btl/base 组件），10.18% 来自

OpenMPI 其他初始化分配（变量注册、进程管理等），仅约 6.41% 来自程序自身的数据结构分配。

- P=4：43.83% 来自同一 MPI 通信缓冲池，14.93% 来自其他 MPI 分配，与 P=1 的分配模式高度相似。
- 两个配置的栈内存消耗均为 1,792 B（约 1.75 KB），证实了程序几乎完全使用堆内存，调用栈深度浅、栈帧尺寸小，符合 FFT 迭代算法的预期。

内存消耗变化趋势分析：
从图1和图2的内存消耗曲线可以看出：

- **初始化阶段（图表左侧上升段）**：程序在 MPI 初始化和首轮内存分配阶段快速消耗内存。P=1 在约 42.66 Mi 指令处即达到峰值，P=4 稍晚（约 46.66 Mi），这源于多进程协调（握手、barrier 同步等）引入的额外指令开销。
- **稳态阶段（图表中间平坦段）**：内存消耗在 FFT 计算过程中保持几乎完全恒定。大量：字符构成的平坦区域说明程序在初始化后不再有显著的净堆分配——数据数组在初始化时一次分配并在多次 FFT 迭代中反复复用，符合高效内存使用的设计原则。偶尔出现的 `@` 标记（详细快照）可能捕获了 `step_mpi` 内部临时向量的分配与释放，但净内存变化极小。
- **进程间对比**：P=1 的单进程峰值总内存为 3.61 MB（3,785,512 B）。P=4 的单个进程峰值内存为 3.53 MB（3,701,712 B），略低于 P=1，这是因为数据块大小从 N=4096（P=1）降至 N/P=1024（P=4），每进程的数据数组尺寸减小。然而 4 个进程的内存消耗总和约为 $4 \times 3.53 = 14.12$ MB，约等于 P=1 的 4 倍，体现了分布式内存模型的典型特征：总内存消耗随进程数线性增长，峰值内存主要受 MPI 运行时框架（约占 50% 以上）而非应用数据的控制。
- **内存组成**：MPI 运行时框架（OpenMPI）占据了峰值内存的 50% 以上，包括通信缓冲区（`opal_free_list_grow_st`）和各种初始化分配。程序自身的 FFT 数据数组及临时向量仅占总内存的小部分。这意味着对于较小的问题规模（如 N=4096），减少 N 并不能显著降低每进程的内存消耗，因为 MPI 框架的固定开销构成了” 内存地板”。

5 实验总结

本实验以 MPI 对快速傅里叶变换（DIF 变体）进行了并行化实现，分别测试了基础通信版本与 MPI_Pack/Unpack 打包优化版本的性能，并利用 Valgrind Massif 分析了内存消耗特征。主要结论如下：

1. **并行加速效果**：在 P=16 时可获得约 1.4 倍加速比，但 P=2,4,8 时均出现不同程度的减速（加速比 0.28–0.77）。P=16 时 1.4 倍的加速比远低于投入 16 倍计算资源的线性加速理想值，根本原因在于当前实现的通信模式：由于当前实现采用固定块状分布，并在每阶段维持"前半区数据位于低编号进程、后半区数据位于高编号进程"的布局方式，因此每个 DIF 阶段都需要跨进程交换蝶形输入数据。这属于当前数据布局与通信策略带来的结果，而非 DIF 算法本身的必然特性。每个 FFT 阶段（共 $\log_2 N$ 个）都需要两次全局集体通信（Sendrecv + Alltoallv），导致通信开销完全吞噬了计算并行化收益。

2. **通信模式的教训**：标准并行 FFT 可通过转置法或 DIT 变体将通信阶段数从 $\log_2 N$ 降至 $\log_2 P$，本实验的实测数据恰好从反面验证了这一原则的必要性——不当的通信设计足以使并行化失去实际价值。

3. **数据打包评估**：MPI_Pack/Unpack 的效果高度依赖于进程数和消息大小，并非普适优化：P=2,4 时一致劣化 5–7%，P=8 时条件性获益 18–22%，P=16 时无明显影响。对于连续内存块的传输场景，打包操作的额外内存拷贝是纯粹的负担。

4. **内存消耗特征**：Valgrind Massif 分析表明每进程峰值内存约 3.5–3.6 MB（N=4096），其中 MPI 运行时框架占 50% 以上。总内存消耗随进程数线性增长，程序在初始化后维持恒定内存占用。